



Report a civic complaint in Hindi, English, or Hinglish — **get it classified, prioritized, and routed to the right department automatically.**

A B.Tech CSE (AI/ML) demo project: a working citizen/officer portal built around a real, trained classification pipeline — not a CPGRAMS replacement, but the same shape of problem.

7

CIVIC DOMAINS

3

LANGUAGES

2

ACCOUNT ROLES

<1s

CLASSIFY TIME



What It Does



Multilingual Intake

Citizens type freely in Devanagari Hindi, English, or Romanized Hindi (Hinglish) — no language picker needed.



5-Stage ML Pipeline

Language detection, text normalization, domain classification, priority scoring, and department routing — end to end.



Priority & SLA

Every complaint gets a Low/Medium/High priority and an automatic SLA clock (48h / 168h / 336h).



Citizen Dashboard

Stat cards, a status donut, and a live table of every complaint a citizen has filed and its progress.



Officer Inbox

Filter by domain/priority/status, update ticket status inline, and see domain/priority/status breakdowns.



Simulated Portal Ref.

Generates a mock government-portal reference per ticket — clearly labeled demo output, not a live integration.



Classification Pipeline

1

Language Detection

Script + lexicon match



2

Normalization

Unicode NFC, noise strip



3

Domain Classification

TF-IDF + LogReg



4

Priority Scoring

TF-IDF + LogReg



5

Dept. Routing

Frozen taxonomy lookup

99.4%

Domain classification test accuracy

98.9%

Priority scoring test accuracy

100%

High-priority recall (test set)

* Measured on a held-out split of the project's synthetic training set (840 train / 180 val / 180 test) — indicative of the pipeline working correctly, not a real-world benchmark.

🔧 WORKED EXAMPLE — LIVE CLASSIFICATION

"मेरे घर के पास बिजली के तार में चिंगारी निकल रही है, बहुत खतरनाक है"
("Sparking wires near my house, it's very dangerous" — typed in Hindi, no language picker used)



LANGUAGE
Hindi

DOMAIN
Electricity

PRIORITY
High

ROUTED TO
Discom

Backend

Python 3.13 FastAPI Uvicorn scikit-learn
TF-IDF + Logistic Regression bcrypt Pydantic

Data Layer

SQLite (local dev) Postgres (production) psycopg2
Dual-backend store.py

Frontend

React 18 (CDN) Babel Standalone Tailwind CSS
Chart.js Single-file, no build step

Deployment & Extras

Render (free web service) Neon / Supabase Postgres
Google OAuth (optional)
Groq / Ollama LLM polish (optional)

Project Structure

```
nagrikmitra-app/  
├─ api.py                # FastAPI app: /predict, /tickets, /auth/*, serves frontend/  
├─ auth_routes.py        # register / login / logout / Google OAuth routes  
├─ nagrikmitra/  
│  ├─ predict.py, preprocess.py, language.py  # the classification pipeline  
│  ├─ train.py, generate_data.py              # model training + synthetic data  
│  ├─ store.py                               # SQLite/Postgres dual-backend data access  
│  ├─ auth.py                               # sessions, password hashing, Google OAuth  
│  └─ taxonomy.py                           # domain → department mapping  
├─ frontend/index.html  # the entire citizen/officer UI (React, single file)  
├─ config/taxonomy.yaml  # frozen domain/priority/SLA rules  
├─ models/*.joblib       # trained domain + priority classifiers  
├─ tests/                 # 52 tests – auth, language, prediction, SLA  
└─ render.yaml            # one-click Render deployment config
```

How To Use It

👤 Citizen

- 1 Register (or continue as guest) and open **Submit**.
- 2 Type the complaint in any supported language.
- 3 Watch it get classified, prioritized & routed live.
- 4 Track every complaint's status under **My Complaints**.

👮 Officer

- 1 Register with the shared officer signup code.
- 2 Land directly on the **Dashboard** (no Submit access).
- 3 Filter the inbox by domain, priority, or status.
- 4 Update a ticket's status inline as it's worked.

Source, docs & live deployment config

Full README, architecture notes, and setup instructions on GitHub:
github.com/devansh-91/nagrikmitra-app

